TD

BANKING ADMIN PORTAL

ARCHITECTURE & WORKFLOW REFERENCE

The whole project, drawn out.

System architecture, layered structure on both sides, request lifecycle, every key user flow, and the audit-recording pipeline — all in one place.

> Every diagram in this document describes a real piece of the codebase you can open and point at. None of it is aspirational — it's all in the repo.

AUTHOR

Piyush Garg

PROJECT

Senior Angular Developer take-home

STACK

Angular 17 · NgRx + Signals · Express

DOCUMENT

Architecture reference v1.0

CONTENTS

Twelve diagrams. One reference.

Read top to bottom for a guided tour from system architecture down to the data model, or jump to whichever section matches the conversation you're having.

Three seams where Signals appear

10 **Correlation-id end-to-end**

One id ties client log to server log to audit

11 **Audit recording pipeline**

No-op writes drop, named actions for status flips

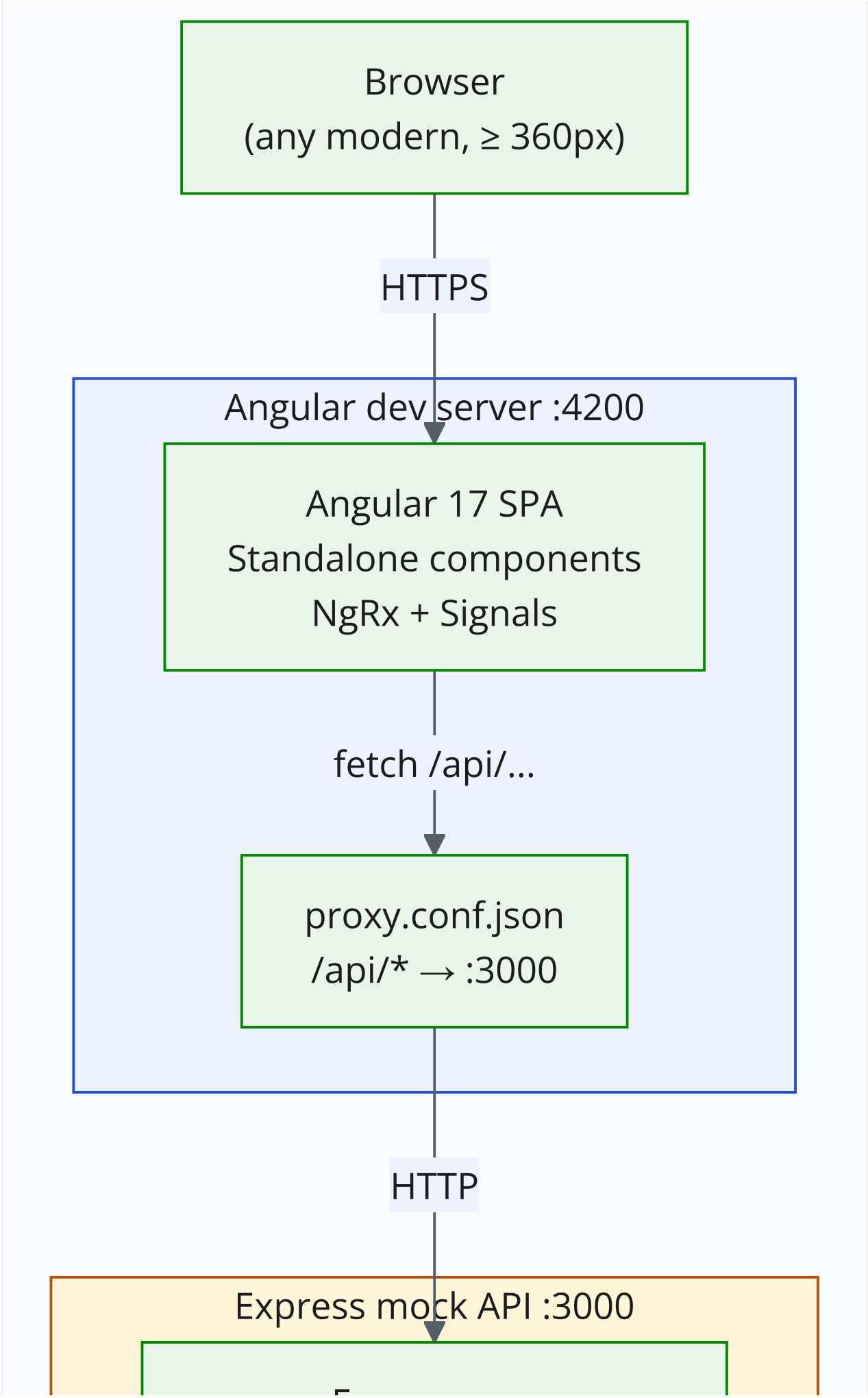
12 **Data model**

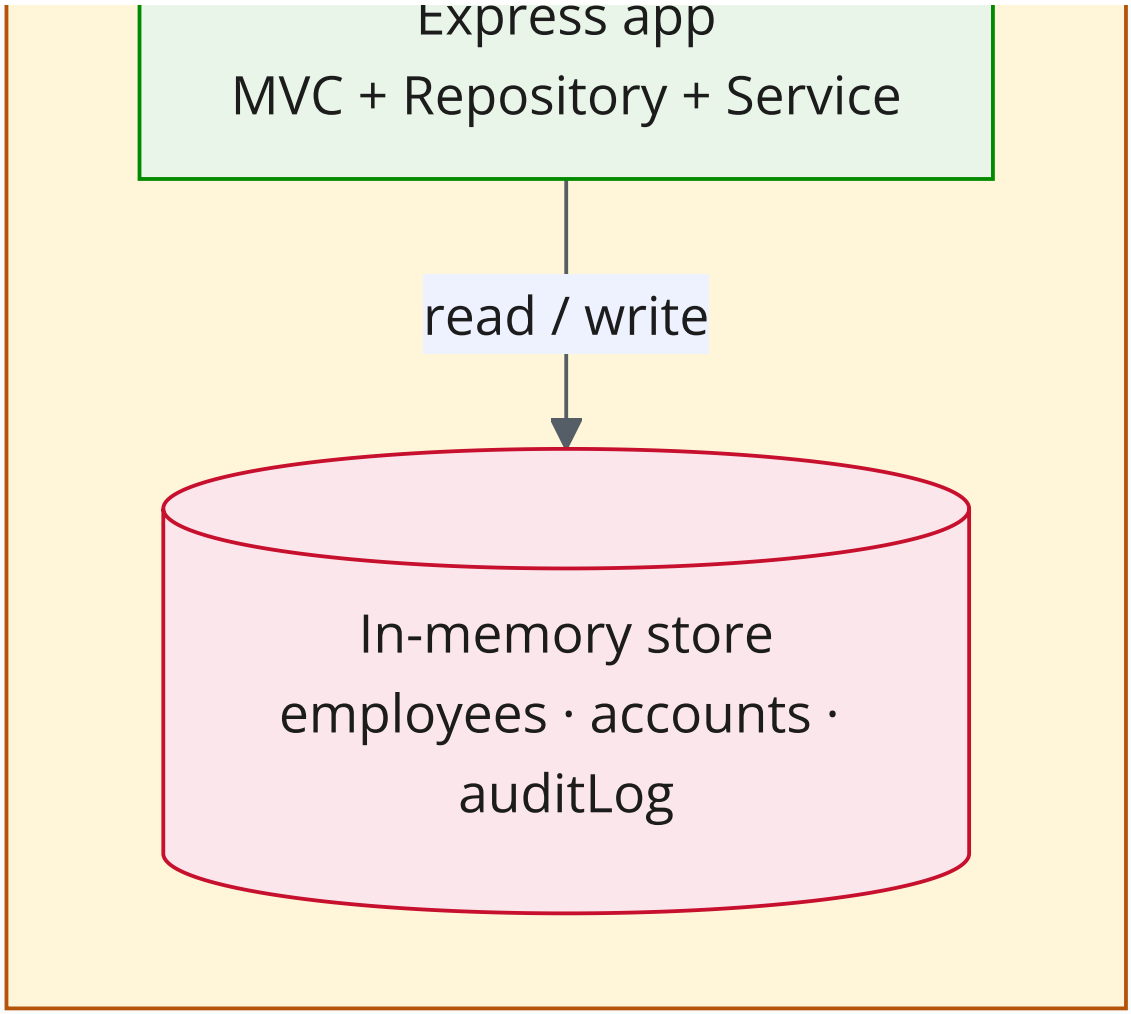
Entity-relationship view of the three tables

SECTION 01 · STRUCTURE

System architecture — the 10 000-ft view

The whole stack on one page. Two processes, one proxy, one in-memory store.


Browser
(any modern, ≥ 360px)
HTTPS
Angular dev server :4200
Angular 17 SPA
Standalone components
NgRx + Signals
fetch /api/...
proxy.conf.json
/api/* → :3000
HTTP
Express mock API :3000

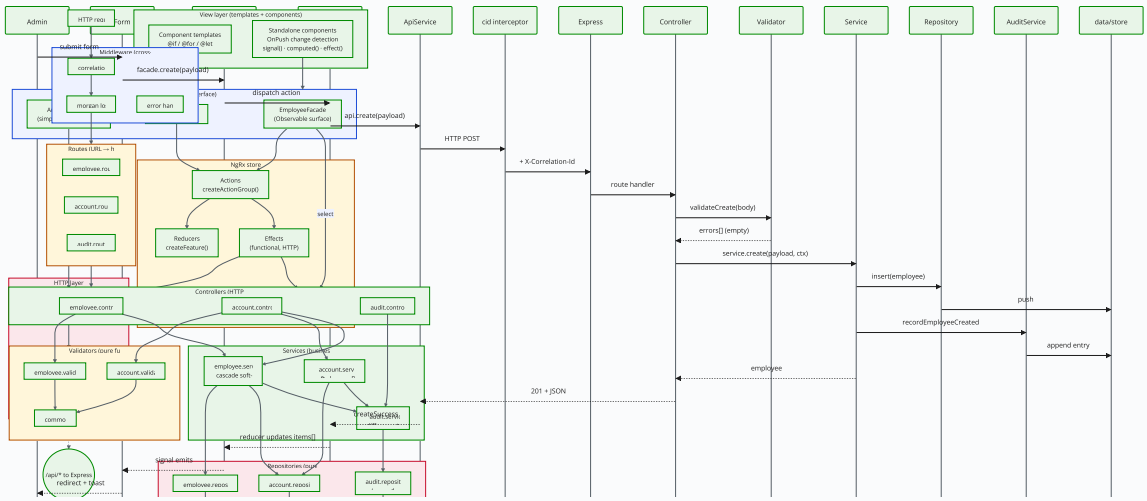


What to notice. *Everything runs locally. No databases, no cloud services, no auth provider. The proxy is what avoids CORS during development; production would terminate at a reverse proxy (nginx, ALB, etc.) instead.*

SECTION 02 · STRUCTURE

Frontend layering

Angular standalone components consume the NgRx store through per-feature facades. Signals appear at the leaf via `toSignal()`. Two HTTP interceptors sit between the services and the network.



What to notice. *Components never import from `Store` directly — they go through a facade. The facade exposes Observables (`items$` , `loading$`); components decide whether to consume them as observables or bridge them to signals via `toSignal()` .*

> **Exception:** the audit log bypasses NgRx entirely — it's read-only, single-view, and a full feature slice would be five files of boilerplate for one GET endpoint.

SECTION 03 · STRUCTURE

Backend layering (MVC + repository + service)

Each layer has exactly one job. The arrow direction is the *call* direction — nothing ever calls upward.

> **The unbreakable rule.** A controller importing a repository, or a service touching `data/store.js` directly, is the architectural smell. None of them do.

SECTION 04 · BEHAVIOUR

Request lifecycle end-to-end

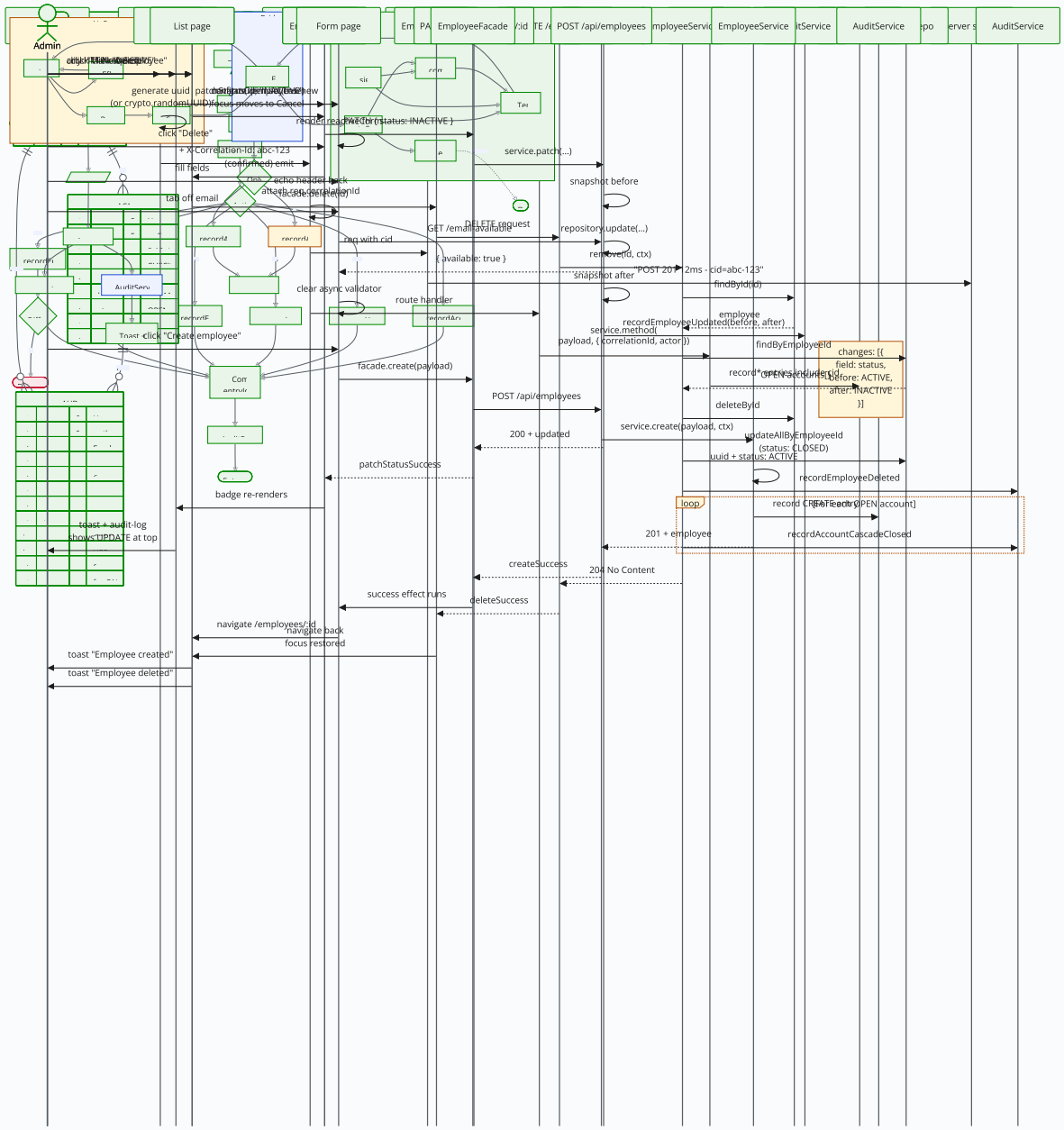
A single `POST /api/employees` request, traced through every box that touches it. Same pattern applies to PUT, PATCH, DELETE.

What this proves. *The single source of truth is the store. The view never holds a copy of the employee — it reads through the facade, which reads the store. Swap the in-memory store for Postgres tomorrow and only the repository changes.*

SECTION 05 · USER FLOW

Creating an employee

From clicking **+ New Employee** to landing on the freshly-created detail page, with the async unique-email check in between.



SECTION 06 · USER FLOW

Toggling employee status (PATCH)

The **Mark INACTIVE** / **Mark ACTIVE** button on the detail page issues a partial update — just the `status` field. The audit trail records a clean `status: ACTIVE → INACTIVE` diff.

SECTION 07 · USER FLOW

Deleting an employee with cascade soft-close

The most complex single action in the system. One DELETE produces multiple audit entries: one `DELETE` for the employee and one `CASCADE_CLOSE` per OPEN account they owned.

The audit trail outlives the deleted row. `GET /api/employees/:id/audit` still resolves even though the row is gone — audit entries are append-only and never depend on a live FK target.

SECTION 08 · USER FLOW

Closing & reopening an account

Status-flip flows demonstrate a non-trivial audit-narrative decision: a CLOSE through the named endpoint produces a `CLOSE` audit action, but the same status change via a generic PATCH produces an `UPDATE` with a diff. The service layer makes the call.

The interesting branch. *`AccountService.patch()` checks whether the incoming patch is only a status change. Multi-field PATCHes always produce `UPDATE`, never the named narrative. This decision lives in the service so the repository stays a dumb append target.*

SECTION 09 · CROSS-CUTTING

State management — NgRx + Signals split

NgRx remains the single source of truth. Signals appear at three specific seams at the component layer.

The boundary. NgRx owns global state. The facade is the public surface. Signals live only inside the component layer. Components dispatch *into* the store via the facade; data flows *out* of the store back via signals.

SECTION 10 · CROSS-CUTTING

Correlation-id end-to-end

Every request carries an `X-Correlation-Id` header so client logs, server logs, and audit entries can be matched up.

Result. *A user reports "the delete button didn't work" → pull the cid from the browser console, grep server logs for that exact cid, see every request/response in that conversation, find the matching audit entry.*

SECTION 11 · CROSS-CUTTING

Audit recording pipeline

Every write in the service layer flows into one of six narrative paths. No-op UPDATEs are dropped so the trail doesn’t fill with noise.

The rule. No-op UPDATEs don’t pollute the trail. Every other action always produces an entry. This rule lives in the service, not the controller, so any path that calls the service inherits it.

SECTION 12 · STRUCTURE

Data model — entity relationships

Three entities. The audit trail is a child of both Employee and Account, but its FKs are *nominal* — they survive deletion of the parent row by design.

Key constraint not visible above. `AUDIT_ENTRY.employeeId` *is an FK in spirit only. When an employee is hard-deleted, the audit entries persist by design — the trail outlives the row. The FK column means "this entry belongs to the audit trail named by this employee id", not "this entry has a live FK target".*

CLOSING

What to take from this

Every diagram in this document describes a real piece of the codebase you can open and point at. The system architecture matches `app.js` + `app.config.ts`; the layered backend matches the folder layout under `server/`; the request lifecycle matches what you see in the Network tab when you click a button.

None of this is aspirational — it's all in the repo. Drawing it on a whiteboard during the interview is a 60-second exercise *because* it was drawn here first.

> *"The interviewer doesn't care how many lines of code I wrote, what frameworks I used, or whether every test passes. They care whether I can stand at a whiteboard, sketch the system, name the alternatives I rejected, point at three specific things I'd change today, and ask better questions than they expected. The code is just the prop. The thinking is the show."*

Companion documents

DOC	FOR
`NGRX_GUIDE.md`	Walking through the state-management story end to end
`ACCESSIBILITY_AUDIT.md`	What WCAG 2.1 AA looks like in this codebase, concretely
`MAX_PAGE_SIZE_CLAMP.md`	One pragmatic security patch with full rationale
`INTERVIEW_REFLECTION.md`	Five-question pre-interview exercise drilled to the speakable answer
`INTERVIEW_PLAYBOOK.md`	Full ten-step interview preparation with role-tailoring matrix